---

1. Назначение ОС и её место в архитектуре вычислительной системы

Операционная система — это программа, которая управляет ресурсами компьютера и служит посредником между железом и пользователем/приложениями.

Два главных назначения:

- Управление ресурсами — распределяет CPU, память, диски, сеть между приложениями
- Абстракция железа — скрывает сложность оборудования за удобным API (системными вызовами)

Место в архитектуре (снизу вверх): Аппаратное обеспечение → Ядро ОС → Системные утилиты и библиотеки → Пользовательские приложения.

---

2. История и эволюция ОС: основные этапы развития

- 1940–50-е: без ОС. Программист управляет машиной вручную, перфокарты, одна задача за раз.
- 1950–60-е: пакетные системы. Задачи выполняются пакетами, появился монитор — прообраз ОС.
- 1960-е: многозадачность. IBM OS/360, Multics — несколько задач в памяти, CPU переключается между ними.
- 1970-е: UNIX. Портируемая ОС на C; концепции файлов, процессов и конвейеров живут по сей день.
- 1980-е: персональные компьютеры. MS-DOS — однозадачная ОС, позже Mac OS и Windows с GUI.
- 1990-е: сети. Windows 95/NT, Linux 1.0 — многозадачность на ПК, работа в интернете.
- 2000-е — сейчас: облако, мобайл, контейнеры. Android, iOS, виртуализация, RTOS.

---

3. Классификация операционных систем

По назначению: общего назначения (Windows, Linux), серверные, мобильные (Android, iOS), встраиваемые, реального времени (FreeRTOS, VxWorks).

По числу пользователей: однопользовательские (MS-DOS) и многопользовательские (UNIX, Linux).

По числу задач: однозадачные (MS-DOS) и многозадачные (Windows, Linux).

По режиму работы: пакетные (мейнфреймы), интерактивные (Linux desktop), реального времени.

По архитектуре ядра: монолитное (Linux), микроядерное (MINIX, QNX), гибридное (Windows NT, macOS). По платформе: x86, ARM, многопроцессорные (SMP), кластерные.

---

4. Архитектура вычислительной системы: взаимодействие аппаратного обеспечения и ОС

Ключевые механизмы взаимодействия:

- Шина (Bus) — общая магистраль, через которую CPU, память и устройства обмениваются данными (PCI-e, USB, SATA).
- Контроллеры устройств — отдельные микросхемы для диска, сети, видео. У каждого есть регистры и буфер.
- Прерывания (IRQ) — устройство сигнализирует CPU «я закончил работу», CPU вызывает обработчик ОС.
- DMA (прямой доступ к памяти) — контроллер сам копирует данные в RAM, не занимая CPU побайтово.
- Драйверы ОС — программные модули, которые «разговаривают» с конкретным контроллером от имени ОС.

---

5. Понятие ядра ОС. Режимы User mode и Kernel mode

Ядро (kernel) — центральная часть ОС, работающая постоянно и управляющая всеми ресурсами: памятью, процессами, устройствами.

У CPU два режима работы:

	User mode (кольцо 3)	**Kernel mode (кольцо 0)**
Кто работает	Обычные приложения	Ядро ОС, драйверы
Доступ к железу	Запрещён напрямую	Полный доступ
Память	Только своё виртуальное пространство	Вся физическая память
Привилегированные инструкции	Нельзя	Можно
Переход	Только через syscall или прерывание	

Зачем: если приложение падает или делает что-то запрещённое — CPU генерирует исключение, ОС завершает только этот процесс, не роняя всю систему.

---

6. Типы архитектур ядер: монолитное, микроядерное, гибридное

Монолитное ядро — весь код ОС в одном пространстве ядра (планировщик, ФС, драйверы, сеть — всё вместе). Плюсы: высокая скорость. Минусы: ошибка в одном модуле роняет всё ядро. Примеры: Linux, UNIX.

Микроядерное — в ядре минимум: IPC, планировщик, базовое управление памятью. Всё остальное — серверы в user space. Плюсы: надёжность и изоляция. Минусы: медленнее из-за частых переключений user↔kernel. Примеры: MINIX, QNX, seL4.

Гибридное — микроядро + часть сервисов возвращена в kernel space ради производительности. Компромисс между скоростью и надёжностью. Примеры: Windows NT, macOS (XNU).

Итого: Монолит = быстро, но хрупко. Микроядро = надёжно, но медленно. Гибрид = золотая середина.

---

7. Системные вызовы: назначение, механизм работы, примеры

Системный вызов (syscall) — единственный легальный способ для программы в user mode попросить ОС сделать что-то привилегированное (читать файл, выделить память и т.д.).

Механизм работы:

1. Приложение кладёт номер вызова и параметры в регистры CPU
2. Выполняет инструкцию syscall / int 0x80 — CPU переключается в Kernel mode
3. Ядро находит нужный обработчик в таблице syscall
4. Ядро выполняет действие
5. Возвращает результат, переключается обратно в User mode

Примеры (Linux): open, read, write, close (файлы); fork, exec, exit, wait (процессы); mmap, brk (память); socket, connect, send, recv (сеть).

8. Прерывания и исключения: различия, роль в работе ОС

Оба механизма заставляют CPU «бросить» текущую задачу и перейти к обработчику ОС.

Прерывание (Interrupt) — источник внешний (асинхронный): диск закончил запись, пришёл сетевой пакет, сработал таймер. Реакция ОС: вызов ISR (interrupt service routine), затем возврат к прерванной задаче.

Исключение (Exception) — источник внутренний (синхронный): текущая инструкция вызвала ошибку (деление на ноль, обращение к несуществующей памяти, привилегированная инструкция в user mode). Реакция ОС: может завершить процесс с ошибкой (Segfault, SIGFPE).

Роль прерываний: без них ОС бы постоянно опрашивала (polling) устройства, тратя 100% CPU впустую. Прерывания позволяют CPU работать, а устройствам — сигнализировать только когда нужно.

9. Понятие процесса и потока. Их различия и области применения

Процесс — запущенная программа со своим изолированным адресным пространством, открытыми файлами, переменными окружения и правами.

Поток (thread) — единица выполнения внутри процесса. Потоки одного процесса делят память и ресурсы, но имеют свой стек и счётчик команд.

	Процесс	Поток
Память	Своя (изолированная)	Общая с другими потоками
Создание	Тяжелее (fork)	Легче
Общение	IPC: pipe, socket, shared memory	Напрямую через общие переменные
Изоляция	Крах одного не роняет другой	Крах может убить весь процесс
Применение	Браузер (вкладки = процессы)	Веб-сервер (запросы = потоки)

10. Состояния процесса и жизненный цикл процесса

- Новый (New) — процесс создаётся, загружается в память
- Готов (Ready) — в памяти, ждёт своей очереди на CPU
- Выполняется (Running) — CPU выполняет инструкции прямо сейчас
- Ожидает (Blocked) — ждёт I/O, семафора или таймера; CPU не использует
- Завершён (Terminated) — процесс закончил работу, ресурсы освобождаются

Переходы: Ready→Running (планировщик выбрал), Running→Blocked (ждёт I/O), Blocked→Ready (I/O завершился), Running→Ready (вытеснение по таймеру).

Дополнительно: состояние Zombie — процесс завершился, но родитель ещё не прочитал его код возврата через wait().

11. Планирование процессорного времени: цели и критерии эффективности

Планировщик (scheduler) — часть ОС, решающая какой процесс получит CPU следующим и на сколько.

Цели планирования:

- Максимальное использование CPU (не давать ему простаивать)
- Справедливость — каждый процесс получает свою долю времени
- Минимальное время отклика для интерактивных задач
- Максимальная пропускная способность (задач в единицу времени)
- Соблюдение дедлайнов для систем реального времени

Метрики эффективности:

- CPU utilization — % времени, когда CPU занят (цель: близко к 100%)
- Throughput — число завершённых задач в единицу времени
- Turnaround time — время от создания процесса до его завершения
- Waiting time — суммарное время в очереди Ready
- Response time — время от запроса до первого отклика

12. Алгоритмы планирования (FIFO, Round Robin, приоритетное)

FCFS/FIFO — кто пришёл первым, тот выполняется до конца. Просто, но страдает «конвой-эффект»: длинная задача блокирует все короткие.

SJF (Shortest Job First) — первым запускается процесс с наименьшим CPU burst. Минимальное среднее время ожидания, но нужно заранее знать длину задачи; возможно голодание длинных задач.

Round Robin — каждый процесс получает квант времени (например 20 мс), потом следующий по кругу. Честно, хорошее время отклика. При маленьком кванте — много переключений контекста; при большом — вырождается в FIFO.

Приоритетное — процессу присваивается приоритет, выполняется тот, у кого он выше. Важные задачи быстрее, но низкоприоритетные могут голодать — решается «старением» (aging): приоритет растёт со временем ожидания.

Многоуровневые очереди — несколько очередей (интерактивные, фоновые, системные) с разными алгоритмами.

Реальные ОС: Linux использует CFS (Completely Fair Scheduler), Windows — многоуровневые очереди с приоритетами.

13. Многозадачность и многопоточность: преимущества и ограничения

Многозадачность — одновременное выполнение нескольких процессов (ОС быстро переключает CPU между ними).

Многопоточность — несколько потоков внутри одного процесса выполняются параллельно (на разных ядрах или псевдопараллельно).

	Многозадачность	**Многопоточность**
Уровень	Несколько процессов	Несколько потоков в одном процессе
Изоляция	Полная	Слабая (общая память)
Накладные расходы	Больше	Меньше

Преимущества:

- Лучшее использование CPU, особенно при нескольких ядрах
- Отзывчивость: пока один поток ждёт I/O, другой работает
- Параллельные вычисления ускоряют обработку данных

Ограничения:

- Race condition — потоки одновременно меняют общие данные, порождая ошибки
- Deadlock — два потока ждут ресурс друг друга и зависают
- Overhead — переключение контекста и синхронизация стоят времени
- GIL в Python — глобальная блокировка не даёт потокам работать по-настоящему параллельно

---

14. Синхронизация процессов и потоков

Синхронизация — механизмы, которые не дают нескольким процессам/потокам одновременно изменять общие данные, что приводит к непредсказуемым результатам.

Проблема: если два потока одновременно делают `counter++`, каждый читает значение, увеличивает и записывает — одно увеличение теряется. Это называется состояние гонки (race condition).

Критическая секция — участок кода, где поток работает с общими данными. Правила: только один поток в критической секции в момент времени; поток не должен бесконечно ждать входа; поток вне критической секции не блокирует других.

Способы синхронизации: мьютексы, семафоры, мониторы, спинлоки, барьеры, условные переменные.

---

15. Примитивы синхронизации: мьютексы, семафоры, спинлоки

Мьютекс (mutex = mutual exclusion) — «замок» на критическую секцию. Только один поток может его захватить (lock), остальные блокируются и ждут. Когда поток выходит — отпускает (unlock). Только тот поток, который захватил мьютекс, может его освободить.

Семафор — счётчик с двумя операциями: wait() (P) — уменьшает счётчик, если он 0 — поток блокируется; signal() (V) — увеличивает счётчик, будит один из ожидающих. Бинарный семафор (0 или 1) = аналог мьютекса. Считающий семафор (0..N) — ограничивает число одновременных потоков (например, максимум 5 потоков в БД).

Спинлок (spinlock) — поток не блокируется, а крутится в цикле «не захвачен ли замок?» (активное ожидание). Плюс: нет переключения контекста. Минус: тратит CPU впустую во время ожидания. Применяется только для очень коротких критических секций в ядре ОС.

Условная переменная (condition variable) — позволяет потоку ждать наступления условия (например, «очередь не пуста»). Используется вместе с мьютексом: поток берёт мьютекс, проверяет условие, если не выполнено — освобождает мьютекс и засыпает, пока другой поток не сделает signal().

---

16. Проблема взаимной блокировки (deadlock): условия возникновения и методы предотвращения

Deadlock — ситуация, когда два или более процесса бесконечно ждут ресурс, который держит другой из них, и никто не может продолжить.

Пример: процесс A держит файл и ждёт принтер; процесс B держит принтер и ждёт файл — оба заблокированы навсегда.

Четыре необходимых условия Коффмана (deadlock возникает только если выполнены все четыре):

1. Взаимное исключение — ресурс использует только один процесс одновременно
2. Удержание и ожидание — процесс держит ресурс и ждёт другой
3. Нет принудительного изъятия — ресурс нельзя отнять, только отдать добровольно
4. Круговое ожидание — цепочка процессов, каждый ждёт ресурс следующего

Методы борьбы:

- Предотвращение (prevention) — устранить одно из четырёх условий. Например, требовать захватывать все ресурсы сразу или устанавливать порядок захвата (нумерация ресурсов).
- Обход (avoidance) — алгоритм банкира: перед выделением ресурса ОС проверяет, останется ли система в «безопасном» состоянии.
- Обнаружение и восстановление (detection & recovery) — ОС периодически строит граф ресурсов, ищет циклы; при обнаружении — убивает один из процессов.

Игнорирование (ostrich algorithm) — в большинстве обычных ОС (Windows, Linux): deadlock редок, проще перезапустить процесс.

17. Классические задачи синхронизации (пример: «обедающие философы»)

Задача «Обедающие философы» (Dining Philosophers):

5 философов сидят за круглым столом. Между каждыми двумя — одна вилка (5 вилок всего). Чтобы есть, философу нужны две вилки (левая и правая). Если все одновременно возьмут левую вилку — deadlock: все ждут правую.

Решения:

- Разрешить брать вилки только если обе свободны (атомарно)
- Один философ берёт вилки в обратном порядке (нарушает круговое ожидание)
- Не более 4 философов садятся одновременно (семафор = 4)
- Мониторы с условными переменными

Задача «Производитель–потребитель» (Producer–Consumer): производитель кладёт данные в буфер, потребитель берёт. Нужно не переполнить буфер и не читать из пустого. Решение: два семафора — empty (свободных мест) и full (занятых мест) + мьютекс на доступ к буферу.

Задача «Читатели–писатели» (Readers–Writers): несколько читателей могут читать одновременно, но писатель работает в одиночку. Решение: мьютекс + счётчик читателей; первый читатель блокирует писателей, последний — разблокирует.

---

18. Назначение подсистемы управления памятью в ОС

Подсистема управления памятью — компонент ОС, отвечающий за распределение физической памяти между процессами и защиту их адресных пространств друг от друга.

Задачи:

- Выделение и освобождение памяти процессам
- Изоляция — процессы не видят память друг друга
- Виртуализация — каждый процесс думает, что у него есть всё адресное пространство целиком
- Подкачка (swap) — временно перемещает страницы на диск при нехватке RAM
- Защита — запрет на запись в чужую или read-only память (иначе Segfault)
- Общая память — контролируемое разрешение нескольким процессам видеть один участок памяти

---

19. Физическая и виртуальная память: основные отличия

Физическая память (RAM) — реальные байты оперативной памяти на плате. Адреса от 0 до размера RAM. Ограничена: например, 16 ГБ.

Виртуальная память — абстракция, которую ОС создаёт для каждого процесса. Каждый процесс видит своё собственное непрерывное адресное пространство (например, от 0 до 2^{64}). На самом деле его страницы разбросаны по физической памяти или лежат на диске (swap).

	Физическая	**Виртуальная**
Размер	Ограничен RAM	Может быть больше RAM
Уникальность	Одна на всех	Своя для каждого процесса
Непрерывность	Фрагментирована	Всегда кажется непрерывной
Доступ	Прямой (ядро)	Через MMU и таблицу страниц

MMU (Memory Management Unit) — аппаратный блок CPU, который на лету переводит виртуальные адреса в физические по таблице страниц.

---

20. Страничная организация памяти

Страничная организация — способ управления памятью, при котором и виртуальная, и физическая память делятся на одинаковые блоки фиксированного размера.

- Страница (page) — блок виртуальной памяти (обычно 4 КБ)
- Фрейм (frame) — блок физической памяти того же размера
- Таблица страниц (page table) — у каждого процесса своя; хранит соответствие «номер страницы → номер фрейма»

Как работает трансляция адреса: виртуальный адрес делится на номер страницы и смещение внутри неё. MMU смотрит в таблицу страниц, находит нужный фрейм, добавляет смещение — получает физический адрес.

TLB (Translation Lookaside Buffer) — кэш последних переводов страниц прямо в CPU. Попадание в TLB — 1 такт; промах — десятки тактов на обход таблицы страниц.

Преимущества: нет внешней фрагментации, простое выделение памяти, легко реализовать swap и общую память.

---

21. Механизмы виртуальной памяти и подкачки (swap)

Виртуальная память позволяет использовать диск как расширение RAM. Страницы, которые давно не использовались, вытесняются на диск в файл подкачки (swap-файл или swap-раздел), освобождая место для активных данных.

Как работает page fault:

1. Процесс обращается к странице, которой нет в RAM (бит present = 0 в таблице страниц)
2. MMU генерирует прерывание — page fault
3. ОС находит нужную страницу на диске
4. Выбирает жертву (страницу для вытеснения) по алгоритму замещения
5. Загружает нужную страницу в освободившийся фрейм, обновляет таблицу
6. Процесс продолжает работу

Thrashing (пробуксовка) — патологическая ситуация: ОС тратит больше времени на подкачку страниц, чем на полезную работу. Происходит при катастрофической нехватке RAM.

Demand paging — страница загружается с диска только в момент первого обращения к ней, а не заранее.

---

22. Алгоритмы замещения страниц

Когда RAM полна и нужно загрузить новую страницу, ОС выбирает жертву для вытеснения на диск.

OPT (оптимальный) — вытесняет страницу, которая дольше всего не будет использоваться в будущем. Оптимален по числу page fault, но нереализуем на практике (нужно знать будущее). Используется как эталон сравнения.

FIFO — вытесняет страницу, которая дольше всего находится в памяти. Просто, но неэффективно: может вытеснить активно используемую страницу.

LRU (Least Recently Used) — вытесняет страницу, которая дольше всего не использовалась. Хорошее приближение к OPT, но требует отслеживания времени обращений. На практике используются аппроксимации.

Clock (алгоритм «часов») — аппроксимация LRU. У каждой страницы есть бит использования (R). «Стрелка» обходит страницы по кругу: если R=1 — сбрасывает в 0 и идёт дальше; если R=0 — вытесняет эту страницу.

NFU / Aging — счётчик для каждой страницы сдвигается вправо, в старший бит добавляется R. Вытесняется страница с наименьшим счётчиком.

---

23. Понятие фрагментации памяти и методы её уменьшения

Фрагментация — потеря использования памяти из-за её дробления на мелкие непригодные кусочки.

Внутренняя фрагментация — выделенный блок памяти больше, чем нужно процессу. Например, процессу нужно 3 КБ, а выделяется страница 4 КБ — 1 КБ теряется внутри. Причина: фиксированный размер блоков выделения.

Внешняя фрагментация — суммарно свободной памяти достаточно, но она разбита на маленькие несмежные куски, ни один из которых не подходит для нового запроса. Возникает при динамическом выделении памяти.

Методы уменьшения:

- Страничная организация — устраняет внешнюю фрагментацию полностью, но добавляет небольшую внутреннюю
- Компактизация (compaction) — ОС перемещает все занятые блоки в одну сторону, собирая свободное пространство. Дорого по времени.
- Slab-аллокатор (в ядре Linux) — заранее выделяет пулы одинаковых объектов, исключая фрагментацию для типовых структур ядра
- Buddy system — память делится на блоки степеней двойки; при освобождении соседние блоки-«близнецы» сливаются обратно

---

24. Управление ресурсами и контроль использования памяти процессами

ОС отслеживает и контролирует, сколько памяти потребляет каждый процесс.

Ключевые механизмы:

Рабочее множество (working set) — набор страниц, которые процесс активно использует в данный момент. ОС старается держать рабочее множество в RAM, чтобы не было thrashing.

Лимиты памяти — ОС или администратор задаёт максимум потребления для процесса (например, cgroups в Linux). При превышении — OOM Killer убивает процесс.

OOM Killer (Out-Of-Memory Killer) — механизм Linux: когда памяти нет совсем, ядро выбирает процесс с наибольшим oom_score (много памяти + низкий приоритет) и убивает его.

Учёт памяти: RSS (Resident Set Size) — страницы реально в RAM; VSZ (Virtual Size) — весь виртуальный адрес включая swap и незагруженные страницы. Смотреть через `top`, `free`, `/proc/meminfo`.

Memory-mapped files (mmap) — файл отображается прямо в адресное пространство процесса; ОС управляет загрузкой страниц файла так же, как обычной памятью.

25. Назначение файловой системы

Файловая система (ФС) — компонент ОС, который организует хранение данных на диске и предоставляет удобный интерфейс для работы с файлами и директориями.

Основные задачи:

- Именование — файлы имеют человекочитаемые имена вместо адресов секторов диска
- Организация — иерархия директорий, пути (например, /home/user/file.txt)
- Хранение метаданных — размер, дата создания/изменения, права доступа, владелец
- Управление пространством — отслеживание свободных и занятых блоков диска
- Защита — разграничение прав (чтение/запись/выполнение для владельца, группы, остальных)
- Надёжность — журналирование (journaling) обеспечивает восстановление после сбоя

Ключевые понятия:

- Inode — структура данных с метаданными файла и адресами его блоков на диске (в UNIX-системах)
- Блок — минимальная единица выделения на диске (обычно 4 КБ)
- Журнал (journal) — лог изменений; при сбое ОС откатывает или доигрывает незавершённые операции

Примеры ФС: ext4, XFS, Btrfs (Linux), NTFS, FAT32, exFAT (Windows), APFS, HFS+ (macOS).

26. Основные компоненты файловой системы

Файловая система состоит из нескольких ключевых структур, которые вместе обеспечивают хранение и поиск данных на диске.

Загрузочный блок (Boot block) — самый первый блок раздела. Содержит загрузчик ОС (bootloader). Есть даже на разделах без ОС, просто пустой.

Суперблок (Superblock) — хранит общую информацию о всей файловой системе: размер раздела, размер блока, число inode, число свободных блоков, тип ФС. Если суперблок повреждён — ФС не монтируется. Поэтому хранится в нескольких копиях.

Inode-таблица — массив inode (index node). Каждый файл и директория имеет свой inode с метаданными: размер, права, владелец, время создания/изменения, адреса блоков данных на диске. Имя файла в inode не хранится — оно в директории.

Блоки данных (Data blocks) — собственно содержимое файлов и директорий.

Bitmap свободных блоков и inode — битовые карты, показывающие какие блоки и inode свободны. ОС смотрит в них при создании нового файла.

Журнал (Journal) — лог незавершённых операций. При сбое ОС смотрит журнал и либо откатывает, либо завершает прерванные операции. Есть в ext4, NTFS, XFS.

27. Типы файловых систем и их особенности (FAT, NTFS, ext4)

FAT32 (File Allocation Table):

- Таблица размещения файлов — цепочка кластеров для каждого файла
- Максимальный размер файла: 4 ГБ (ограничение 32-битного счётчика)
- Максимальный раздел: 2 ТБ
- Нет прав доступа, нет журнала, нет шифрования
- Плюс: поддерживается везде (флешки, фотоаппараты, TV)
- exFAT — современная замена FAT32: снимает лимит 4 ГБ, для флешек и SD-карт

NTFS (New Technology File System, Windows):

- Журналируемая ФС с MFT (Master File Table) — аналог inode-таблицы
- Максимальный размер файла и раздела: практически не ограничен (16 ЭБ теоретически)
- Поддерживает: права доступа (ACL), шифрование (EFS), сжатие, жёсткие и символьные ссылки, потоки данных (ADS)
- Надёжна: журнал восстанавливает ФС после сбоя
- Минус: слабая нативная поддержка в Linux/macOS

ext4 (Linux):

- Журналируемая ФС, стандарт для Linux
- Максимальный файл: 16 ТБ, раздел: 1 ЭБ
- Поддерживает: extents (непрерывные блоки вместо цепочек), delayed allocation, журналирование
- Обратно совместима с ext2/ext3
- Надёжна, быстра, хорошо изучена
- Btrfs / XFS / ZFS — современные ФС с COW (copy-on-write), снапшотами, встроенным RAID и проверкой целостности данных.

28. Организация доступа к файлам и каталогам

Доступ к файлу проходит через несколько уровней:

Путь и разрешение имени: ОС разбирает путь (/home/user/file.txt) по компонентам. Для каждого компонента читает директорию, находит имя → inode, переходит к следующему уровню.

Директория — это специальный файл, содержащий таблицу «имя файла → номер inode». Сам inode хранит метаданные и адреса блоков с данными.

Методы доступа к данным файла:

- Последовательный — читать байт за байтом с начала (текстовые файлы, потоки)
- Произвольный (random access) — lseek() позволяет прыгнуть на любую позицию (базы данных, бинарные файлы)
- Memory-mapped (mmap) — файл отображается в адресное пространство; доступ как к обычному массиву в памяти

Дескриптор файла (file descriptor) — целое число (0, 1, 2...), которое ОС выдаёт при открытии файла через open(). Все дальнейшие операции (read, write, close) идут через него. 0 = stdin, 1 = stdout, 2 = stderr.

Жёсткая ссылка (hard link) — несколько имён указывают на один inode. Файл удаляется только когда счётчик ссылок падает до 0.

Символьная ссылка (symlink) — отдельный файл, содержащий путь к другому файлу. Аналог ярлыка в Windows.

29. Права доступа и модели безопасности файловых систем

UNIX/Linux — модель DAC (Discretionary Access Control):

Каждый файл имеет три набора прав для трёх категорий:

- Владелец (owner) / Группа (group) / Остальные (others)
- Права: r (чтение=4), w (запись=2), x (выполнение=1)

Пример: `-rwxr-xr--` означает: владелец — rwx (7), группа — r-x (5), остальные — r-- (4) → chmod 754.

Специальные биты: SUID — файл выполняется с правами владельца (например, /usr/bin/passwd); SGID — с правами группы; Sticky bit — только владелец может удалить свой файл в общей директории (/tmp).

Windows NTFS — модель ACL (Access Control List):

У каждого файла/папки есть список ACE (Access Control Entry): кому (пользователь или группа) и что разрешено или запрещено (чтение, запись, выполнение, изменение прав и т.д.). Более гибко чем UNIX: можно задать права для конкретного пользователя и даже явно запретить.

SELinux / AppArmor — MAC (Mandatory Access Control): принудительная политика безопасности поверх DAC. Даже root ограничен политиками. Применяется на серверах и в Android.

30. Кэширование и буферизация ввода-вывода

Диск в сотни раз медленнее RAM. Чтобы не ждать диск на каждой операции, ОС использует кэш и буфер.

Page cache (кэш страниц) — ОС держит в RAM копии недавно прочитанных блоков диска. При повторном чтении — отдаёт из кэша, не обращаясь к диску. При записи — сначала пишет в кэш, на диск сбрасывает позже (write-back).

Write-back vs Write-through:

- Write-through — запись идёт сразу на диск и в кэш. Надёжно, но медленно.
- Write-back — запись идёт в кэш, на диск сбрасывается периодически (dirty pages). Быстро, но при сбое данные могут потеряться.

Буферизация ввода-вывода — промежуточный буфер в памяти накапливает данные перед записью или чтением большими блоками (эффективнее, чем писать по 1 байту).

Readahead (упреждающее чтение) — ОС замечает, что файл читается последовательно, и заранее подгружает следующие блоки в кэш — до того как программа их запросила.

fsync() / fdatasync() — системные вызовы, которые принудительно сбрасывают dirty pages на диск. Нужны для баз данных и критически важных данных.

31. Понятие RAID и его основные уровни

RAID (Redundant Array of Independent Disks) — технология объединения нескольких физических дисков в один логический для повышения производительности и/или надёжности.

RAID 0 (Striping — чередование):

- Данные делятся на полосы и пишутся поочерёдно на все диски
- Скорость: максимальная (все диски работают параллельно)
- Надёжность: нулевая — выход одного диска уничтожает все данные
- Применение: видеомонтаж, временные данные, где важна скорость

RAID 1 (Mirroring — зеркалирование):

- Одни и те же данные пишутся на два диска одновременно
- Надёжность: высокая — один диск можно потерять без потери данных
- Скорость чтения: выше (читать можно с любого); записи: как у одного диска
- Применение: системный раздел, критически важные данные

RAID 5 (Striping + паритет):

- Данные и блок чётности (parity) распределяются по всем дискам
- Минимум 3 диска; выдерживает отказ любого одного
- Ёмкость: (N-1) дисков из N
- Применение: серверы, баланс надёжности и ёмкости

RAID 6 — как RAID 5, но два блока паритета: выдерживает отказ двух дисков одновременно.
RAID 10 (1+0) — зеркала из пар дисков, объединённые в stripe. Высокая скорость + высокая надёжность. Требует минимум 4 диска.

32. Архитектура операционной системы Windows (HAL, ядро NT)

Windows NT — гибридная ОС. Архитектура строится послойно:

HAL (Hardware Abstraction Layer) — самый нижний слой. Скрывает различия между конкретным железом (разные материнские платы, чипсеты) от остального ядра. Благодаря HAL одно и то же ядро работает на разном оборудовании.

Ядро NT (ntoskrnl.exe) — центральный компонент: планировщик потоков, управление памятью, синхронизация, обработка прерываний и исключений. Работает в Kernel mode.

Исполнительная система (Executive) — набор менеджеров внутри ядра: менеджер процессов, менеджер виртуальной памяти, менеджер I/O, реестра, безопасности, объектов. Тоже в Kernel mode.

Драйверы устройств — работают в Kernel mode, взаимодействуют с железом через HAL.

Win32 подсистема (csrss.exe) — обеспечивает Win32 API для приложений. Работает частично в user mode.

Пользовательские приложения (User mode) — обычные программы, системные утилиты, антивирусы. Взаимодействуют с ядром только через системные вызовы (у Windows они называются Native API / ntdll.dll).

33. Роль реестра Windows и его структура

Реестр (Registry) — централизованная иерархическая база данных, где Windows хранит настройки ОС, приложений и оборудования.

Зачем нужен: заменяет разрозненные .ini-файлы из старых Windows; единое место для конфигурации системы, драйверов, пользовательских предпочтений.

Структура реестра — дерево из кустов (hives), ключей (keys) и значений (values):

- HKEY_LOCAL_MACHINE (HKLM) — настройки всей системы и оборудования: драйверы, установленные программы, конфигурация ОС. Не зависит от пользователя.
- HKEY_CURRENT_USER (HKCU) — настройки текущего вошедшего пользователя: тема, обои, параметры приложений.
- HKEY_USERS (HKU) — профили всех пользователей системы.
- HKEY_CLASSES_ROOT (HKCR) — ассоциации файлов (какая программа открывает .pdf, .mp3) и COM-объекты.
- HKEY_CURRENT_CONFIG — текущая конфигурация оборудования.

Физически реестр хранится в файлах-кустах: SYSTEM, SOFTWARE, SAM, SECURITY, NTUSER.DAT — в папке C:\Windows\System32\config.

Инструменты: regedit.exe — графический редактор; reg.exe — работа с реестром из командной строки.

34. Процессы и службы в Windows

Процессы в Windows — аналог UNIX: у каждого своё виртуальное адресное пространство, токен безопасности, дескрипторы объектов. Создаются через CreateProcess(), а не fork().

Системные процессы (всегда запущены):

- System — ядро NT, драйверы
- smss.exe — Session Manager, запускает подсистемы
- csrss.exe — Win32 подсистема
- wininit.exe, winlogon.exe — инициализация и вход в систему
- lsass.exe — служба авторизации и безопасности
- svchost.exe — хост для множества служб Windows (один процесс держит несколько служб)

Службы Windows (Services) — аналог демонов в Linux. Работают в фоне, не требуют входа пользователя, могут стартовать автоматически при загрузке. Примеры: Windows Update, DHCP Client, Print Spooler, Defender.

Управление службами:

- services.msc — графический менеджер служб
- sc.exe — управление из командной строки: `sc start/stop/query имя`
- Параметры запуска: Automatic (при загрузке), Manual (по требованию), Disabled

Типы учётных записей служб: LocalSystem (максимальные права), NetworkService, LocalService (ограниченные права).

35. Средства мониторинга процессов (Task Manager, Process Explorer)

Task Manager (taskmgr.exe) — встроенный диспетчер задач Windows. Вкладки: Processes (все процессы с CPU/RAM/Disk/Network), Performance (графики использования ресурсов), App History, Startup (автозагрузка), Services, Users. Позволяет завершить процесс, изменить приоритет, открыть расположение файла.

Process Explorer (Sysinternals, Microsoft) — расширенная альтернатива Task Manager:

- Показывает дерево процессов (какой процесс кем запущен)
- Детальная информация: открытые дескрипторы, DLL, потоки, сетевые соединения
- Цветовая маркировка: синий = процессы пользователя, розовый = службы, красный = завершается
- Проверка подписи процесса через VirusTotal прямо из интерфейса
- Заменяет Task Manager полностью

Process Monitor (procmon) — отслеживает в реальном времени все операции процессов: обращения к файлам, реестру, сети, процессам. Незаменим при отладке и расследовании проблем.

Другие полезные инструменты Sysinternals:

- Autoruns — вся автозагрузка системы
- TCPView — все активные сетевые соединения
- Handle — какой процесс держит открытый файл

  Командная строка: `tasklist` — список процессов; `taskkill /PID 1234` — завершить процесс.

36. Командная строка и терминальные средства Windows

CMD (cmd.exe) — классическая командная строка Windows, наследник MS-DOS. Поддерживает базовые команды: dir, cd, copy, del, ren, mkdir, ipconfig, ping, netstat, tasklist, taskkill, reg, sc. Возможности скриптования через .bat/.cmd файлы ограничены.

Windows Terminal — современный терминал (с Windows 10/11), объединяет CMD, PowerShell и WSL в одном окне с вкладками. Поддерживает цвета, шрифты, профили.

Полезные команды CMD:

- `ipconfig /all` — сетевые настройки
- `netstat -an` — активные соединения
- `sfc /scannow` — проверка системных файлов
- `chkdsk C: /f` — проверка диска
- `systeminfo` — информация о системе
- `gpupdate /force` — обновить групповые политики

WSL (Windows Subsystem for Linux) — позволяет запускать полноценное Linux-окружение (Ubuntu, Debian и др.) прямо внутри Windows. WSL 2 использует реальное ядро Linux. Можно работать с bash, apt, grep, vim, docker.

Переменные среды: %PATH%, %TEMP%, %USERPROFILE%, %SYSTEMROOT% — задаются через sysdm.cpl → Дополнительно → Переменные среды.

37. PowerShell как средство автоматизации администрирования

PowerShell — мощная оболочка и язык скриптов от Microsoft. В отличие от CMD, работает не с текстом, а с объектами .NET. Доступна на Windows, Linux и macOS (PowerShell Core).

Ключевые концепции:

Командлеты (Cmdlets) — встроенные команды в формате Глагол-Существительное: Get-Process, Stop-Service, Set-Item, New-Item, Remove-Item, Get-Content, Write-Output.

Конвейер (Pipeline) — передаёт объекты (не текст) между командлетами: `Get-Process | Where-Object {$_.CPU -gt 10} | Sort-Object CPU -Descending`

Переменные: `$name = "World"`, `$processes = Get-Process`

Скрипты (.ps1) — файлы с последовательностью команд. Политика выполнения (ExecutionPolicy) контролирует разрешение на запуск: Set-ExecutionPolicy RemoteSigned. Полезные примеры:

- `Get-Service | Where-Object {$_.Status -eq "Running"}` — все запущенные службы
- `Get-EventLog -LogName System -Newest 50` — последние 50 событий системного журнала
- `Restart-Computer -Force` — перезагрузка
- `Get-ChildItem -Recurse -Filter *.log` — найти все .log файлы рекурсивно
- `Invoke-WebRequest -Uri "https://example.com"` — HTTP-запрос

Модули расширяют возможности: ActiveDirectory, Exchange, Azure, VMware — администрирование целых инфраструктур через PowerShell.

---

38. Сценарии PowerShell и области их применения

Сценарий PowerShell (.ps1) — текстовый файл с последовательностью команд, переменных, условий и циклов. Это полноценный язык программирования для автоматизации задач администрирования.

Основные конструкции языка:

Условия: `if ($x -gt 10) { ... } elseif (...) { ... } else { ... }`

Циклы: `foreach ($item in $collection) { ... }` и `for`, `while`, `do-while`

Функции: `function Get-MyInfo { param($Name) Write-Output "Hello $Name" }`

Обработка ошибок: `try { ... } catch { Write-Error $_.Exception.Message } finally { ... }`

Параметры скрипта: `param([string]$Server, [int]$Port = 80, [switch]$Verbose)`

Области применения:

Администрирование Windows — создание пользователей, управление группами, настройка политик, работа с реестром, управление службами и дисками.

Active Directory — `New-ADUser`, `Get-ADGroupMember`, `Disable-ADAccount` — массовые операции над сотнями учёток в несколько строк.

Мониторинг и отчёты — сбор данных о состоянии серверов, дисковом пространстве, загрузке CPU, отправка отчётов на почту.

Работа с файлами и данными — массовое переименование, парсинг CSV/JSON/XML, генерация отчётов в Excel через COM-объекты.

Сеть и безопасность — настройка файрвола (New-NetFirewallRule), управление сертификатами, сканирование портов.

CI/CD и DevOps — скрипты сборки, деплоя, тестирования в Azure DevOps, GitHub Actions.

Облако (Azure/AWS) — модули Az и AWS Tools позволяют управлять облачной инфраструктурой полностью через PowerShell.

Планировщик задач — скрипты запускаются по расписанию через Task Scheduler: `Register-ScheduledTask`.

---

39. Архитектура ОС Linux

Linux — операционная система с монолитным ядром, построенная по принципам UNIX. Архитектура состоит из нескольких слоёв:

Аппаратное обеспечение — CPU, RAM, диски, сетевые карты, периферия.

Ядро Linux (kernel) — работает в Kernel mode, управляет всеми ресурсами. Состоит из подсистем:

- Управление процессами и планировщик (CFS)
- Управление памятью (VMM, страничная организация)
- Виртуальная файловая система (VFS)
- Сетевой стек (TCP/IP)
- Драйверы устройств
- Подсистема IPC
- Подсистема безопасности (SELinux, AppArmor, capabilities)

Системные вызовы (syscall interface) — граница между Kernel mode и User mode. Около 300+ системных вызовов: read, write, fork, mmap, socket и т.д.

Стандартная библиотека C (glibc) — обёртка над системными вызовами для программ. Функции open(), malloc(), printf() внутри вызывают соответствующие syscall.

Пользовательское пространство (User space):

- Системные демоны (systemd, cron, sshd)
- Командные оболочки (bash, zsh)
- Утилиты (ls, grep, awk, sed)
- Прикладные программы

Особенности Linux: монолитное ядро с поддержкой загружаемых модулей (LKM), открытый исходный код, портируемость (x86, ARM, RISC-V, MIPS и др.), огромное число поддерживаемых файловых систем и устройств.

---

40. Особенности монолитного ядра Linux

Монолитное ядро — весь код ОС (планировщик, управление памятью, ФС, драйверы, сеть) выполняется в едином адресном пространстве Kernel mode.

Ключевые особенности ядра Linux:

Загружаемые модули ядра (LKM — Loadable Kernel Modules) — главное отличие от «чистого» монолита. Модули (драйверы, ФС, сетевые протоколы) можно подгружать и выгружать в работающее ядро без перезагрузки. Команды: `lsmod` (список модулей), `modprobe имя` (загрузить), `rmmod имя` (выгрузить), `modinfo имя` (информация).

Производительность — компоненты общаются напрямую через вызовы функций внутри ядра, без переключений контекста. Это быстрее микроядра.

Единое адресное пространство — все подсистемы ядра работают в одном пространстве, имеют прямой доступ к структурам друг друга. Плюс: скорость. Минус: ошибка в одном модуле (например, плохой драйвер) может уронить всё ядро (Kernel panic).

Версионирование и стабильность ABI — ядро Linux намеренно не гарантирует стабильный внутренний ABI (двоичный интерфейс) между версиями. Модули нужно перекомпилировать под каждую версию ядра.

Конфигурируемость — при компиляции ядра можно включать/выключать тысячи параметров (make menuconfig). Можно собрать минимальное ядро для встраиваемых систем или максимально функциональное для сервера.

Планировщик CFS (Completely Fair Scheduler) — стандартный планировщик Linux с O(log n) сложностью, использует красно-чёрное дерево для справедливого распределения CPU.

---

41. Виртуальная файловая система (VFS) в Linux

VFS (Virtual File System) — абстрактный слой в ядре Linux между системными вызовами и конкретными реализациями файловых систем.

Зачем нужна: программа вызывает `open("/home/file.txt")` — ей не важно, лежит файл на ext4, NTFS, NFS (по сети) или в tmpfs (в памяти). VFS принимает вызов и перенаправляет к нужному драйверу ФС.

Ключевые абстракции VFS:

Суперблок (superblock) — объект, представляющий смонтированную файловую систему. Хранит метаданные всей ФС и указатели на операции с ней.

Inode — объект, представляющий файл или директорию. Содержит метаданные (права, размер, время) и указатель на операции (inode_operations): создание, удаление, поиск имён.

Dentry (directory entry) — кэшированное соответствие «имя → inode». Dentry cache (dcache) ускоряет разрешение путей: повторное обращение к тому же пути не требует обхода директорий на диске.

File — объект открытого файла, связан с конкретным процессом. Хранит текущую позицию (offset) и указатели на операции (file_operations): read, write, seek, mmap.

Как работает open("/etc/passwd"):

1. VFS разбирает путь через dcache/dentry
2. Для каждого компонента пути находит inode
3. Проверяет права доступа
4. Создаёт объект file, возвращает дескриптор процессу

Монтирование: `mount /dev/sda1 /mnt/data -t ext4` — подключает ФС к дереву каталогов. VFS встраивает суперблок новой ФС в нужную точку.

---

42. Файловая система Linux и иерархия каталогов

В Linux всё — файл (файлы, директории, устройства, сокеты, процессы). Иерархия описана стандартом FHS (Filesystem Hierarchy Standard).

Основные каталоги:

- `/` — корень, начало всего дерева каталогов
- `/bin` — базовые утилиты пользователя: ls, cp, mv, cat, bash (нужны при загрузке)
- `/sbin` — системные утилиты администратора: fdisk, ifconfig, fsck
- `/etc` — конфигурационные файлы всей системы: /etc/passwd, /etc/fstab, /etc/hosts, /etc/ssh/
- `/home` — домашние директории пользователей: /home/ivan/, /home/maria/
- `/root` — домашняя директория суперпользователя root
- `/var` — изменяемые данные: логи (/var/log/), почта, базы данных пакетного менеджера, кэши
- `/tmp` — временные файлы; очищается при перезагрузке
- `/usr` — пользовательские программы и библиотеки: /usr/bin/, /usr/lib/, /usr/share/
- `/lib` — разделяемые библиотеки (.so), нужные для /bin и /sbin
- `/dev` — файлы устройств: /dev/sda (диск), /dev/null, /dev/zero, /dev/random
- `/proc` — виртуальная ФС; информация о процессах и ядре: /proc/1234/ — данные процесса с PID 1234, /proc/meminfo, /proc/cpuinfo
- `/sys` — виртуальная ФС; информация об оборудовании и драйверах (sysfs)
- `/mnt`, `/media` — точки монтирования внешних устройств и разделов
- `/boot` — ядро Linux (vmlinuz), initrd, загрузчик GRUB
- `/opt` — опциональные сторонние приложения

---

43. Права доступа в Linux (chmod, chown, umask)

Каждый файл имеет владельца (user), группу (group) и три набора прав — для владельца, группы и остальных (others).

Права: r (read=4), w (write=2), x (execute=1). Показываются командой `ls -l`: `-rwxr-xr-- 1 ivan developers 4096 Jun 1 file.sh` Тип файла + права владельца (rwx) + права группы (r-x) + права остальных (r--)

chmod — изменить права:

- Числовой режим: `chmod 755 file.sh` → rwxr-xr-x
- Символьный режим: `chmod u+x file.sh` (добавить x владельцу), `chmod go-w file` (убрать w у группы и остальных), `chmod a+r file` (добавить r всем)

chown — изменить владельца и/или группу:

- `chown ivan file.txt` — сменить владельца
- `chown ivan:developers file.txt` — сменить владельца и группу
- `chown -R ivan /home/ivan/` — рекурсивно для всей директории

umask — маска прав по умолчанию при создании новых файлов и директорий. Вычитается из максимальных прав: файлы 666, директории 777. При umask 022: файлы создаются с правами 644 (rw-r--r--), директории с 755 (rwxr-xr-x). Посмотреть: `umask`; задать: `umask 027`.

Специальные биты:

- SUID (4): файл запускается с правами владельца. `chmod u+s /usr/bin/passwd`
- SGID (2): файл запускается с правами группы; для директории — новые файлы наследуют группу
- Sticky bit (1): только владелец может удалить свой файл. Применяется для /tmp: `chmod +t /tmp`

---

44. Командная оболочка Bash и её возможности

Bash (Bourne Again SHell) — стандартная оболочка в большинстве дистрибутивов Linux. Это одновременно интерактивный терминал и язык скриптования.

Переменные: `name="Linux"`, `echo $name`, `echo ${name}`. Переменные окружения: `export PATH=$PATH:/opt/bin`.

Специальные переменные: `$0` — имя скрипта, `$1..$9` — аргументы, `$#` — число аргументов, `$?` — код возврата последней команды, `$$` — PID текущего процесса, `$@` — все аргументы.

Конвейер и перенаправление:

- `cmd1 | cmd2` — вывод cmd1 → ввод cmd2
- `cmd > file` — перенаправить stdout в файл (перезаписать)
- `cmd >> file` — добавить в конец файла
- `cmd 2> err.log` — перенаправить stderr
- `cmd &> file` — stdout и stderr вместе
- `cmd < file` — взять stdin из файла

Условия и циклы:

```
if [ $x -gt 10 ]; then echo "big"; fi
for f in *.txt; do echo $f; done
while read line; do echo $line; done < file.txt
```

Функции: `myfunc() { echo "arg: $1"; }`

Подстановка команд: `files=$(ls /tmp)`, `echo "Today: $(date)"`

Арифметика: `echo $((2 + 2))`, `(( count++ ))`

Полезные сочетания: `&&` (выполнить если предыдущая успешна), `||` (если ошибка), `;` (выполнить последовательно).

Горячие клавиши: Ctrl+R — поиск в истории, Ctrl+C — прервать, Ctrl+Z — приостановить, Tab — автодополнение.

---

45. Управление процессами в Linux (fork, exec, nice, kill)

fork() — системный вызов создания нового процесса. Дочерний процесс — точная копия родителя (память, открытые файлы, окружение). Возвращает: 0 в дочернем, PID дочернего в родительском. Используется механизм COW (Copy-On-Write) — реальное копирование памяти происходит только при записи.

exec() — семейство вызовов (execve, execl, execvp): заменяет образ текущего процесса новой программой. PID остаётся тем же. Типичная схема в shell: fork() → в дочернем exec() новой команды → родитель wait().

wait() / waitpid() — родитель ждёт завершения дочернего и забирает его код возврата. Если не вызвать — дочерний становится зомби (zombie).

nice и приоритеты: nice — значение от -20 (наивысший приоритет) до +19 (наинизший). Обычные процессы стартуют с nice=0.

- `nice -n 10 ./script.sh` — запустить с пониженным приоритетом
- `renice -n 5 -p 1234` — изменить приоритет существующего процесса
- Только root может задавать отрицательные значения (повышать приоритет)

kill — посылает сигнал процессу: `kill -SIGTERM 1234` (мягкое завершение), `kill -SIGKILL 1234` (принудительное, нельзя игнорировать), `kill -SIGHUP 1234` (перечитать конфигурацию).

Полезные команды:

- `ps aux` — все процессы
- `pstree` — дерево процессов
- `top` / `htop` — интерактивный мониторинг
- `bg` / `fg` — перевести процесс в фон/передний план
- `jobs` — список фоновых задач текущего shell
- `nohup cmd &` — запустить процесс, который не умрёт при выходе из терминала

---

46. Межпроцессное взаимодействие (IPC): каналы, сигналы, сокеты

IPC (Inter-Process Communication) — механизмы, позволяющие процессам обмениваться данными и синхронизироваться.

Каналы (Pipes):

- Анонимный канал `|` — однонаправленный, только между родственными процессами: `ls | grep txt`
- Именованный канал (FIFO) — файл в ФС, через который общаются любые процессы: `mkfifo /tmp/mypipe`; один пишет, другой читает

Сигналы (Signals) — асинхронные уведомления процессу:

- SIGTERM (15) — запрос завершения (можно поймать и обработать)
- SIGKILL (9) — принудительное убийство (нельзя поймать или игнорировать)
- SIGINT (2) — Ctrl+C из терминала
- SIGHUP (1) — терминал закрыт; демоны используют для перечитки конфига
- SIGCHLD — дочерний процесс завершился
- SIGSEGV — обращение к недопустимой памяти (Segmentation fault)

Разделяемая память (Shared Memory) — самый быстрый IPC: несколько процессов маппят один участок физической памяти. Требует синхронизации (семафоры). API: `shmget`, `shmat` (System V) или `mmap` с MAP_SHARED.

Семафоры (Semaphores) — используются для синхронизации при работе с разделяемой памятью. System V: `semget`, `semop`; POSIX: `sem_open`, `sem_wait`, `sem_post`.

Очереди сообщений (Message Queues) — процессы обмениваются структурированными сообщениями через очередь в ядре. `mq_open`, `mq_send`, `mq_receive`.

Сокеты (Sockets):

- Unix Domain Sockets (UDS) — сокеты в файловой системе для IPC на одной машине. Быстрее сетевых. Пример: /var/run/docker.sock
- TCP/UDP сокеты — сетевое взаимодействие между процессами на разных машинах

---

47. Многопоточность в Linux (pthreads)

Linux реализует потоки через clone() — системный вызов, создающий «лёгкий процесс» (LWP), разделяющий с родителем адресное пространство, файловые дескрипторы и другие ресурсы.

POSIX Threads (pthreads) — стандартный API для работы с потоками в C/C++ на Linux:

Создание потока: `pthread_create(&tid, NULL, func, arg)` — запускает функцию func в новом потоке.

Ожидание завершения: `pthread_join(tid, &retval)` — ждёт поток и получает результат.

Завершение: `pthread_exit(retval)` — завершает текущий поток.

Мьютексы в pthreads:

```
pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER;
pthread_mutex_lock(&m);
// критическая секция
pthread_mutex_unlock(&m);
```

Условные переменные:

```
pthread_cond_t cond = PTHREAD_COND_INITIALIZER;
pthread_cond_wait(&cond, &mutex);   // ждать условия
pthread_cond_signal(&cond);          // разбудить один поток
pthread_cond_broadcast(&cond);       // разбудить все потоки
```

Атрибуты потоков: можно задать размер стека, политику планирования (SCHED_FIFO, SCHED_RR, SCHED_OTHER), detach state.

Потокобезопасность: функции, которые можно безопасно вызывать из нескольких потоков одновременно. Многие функции стандартной библиотеки имеют потокобезопасные версии (суффикс _r): strtok_r, localtime_r.

Thread-local storage (TLS): `__thread int counter;` — у каждого потока своя копия переменной.

---

48. Systemd и управление службами

Systemd — система инициализации и менеджер служб, стандарт для большинства современных дистрибутивов Linux (Ubuntu, Fedora, Debian, RHEL). Заменил SysV init и Upstart.

Основная идея: systemd управляет не только запуском служб, но и монтированием, сетью, логированием, таймерами, сокетами — всей инициализацией системы.

Unit-файлы — конфигурация всего в systemd. Типы юнитов:

- .service — служба (демон)
- .socket — сокет для активации службы по запросу
- .timer — планировщик (замена cron)
- .mount — точка монтирования
- .target — группа юнитов (аналог runlevel)

Структура .service файла:

```
[Unit]
Description=My Service
After=network.target

[Service]
ExecStart=/usr/bin/myapp
Restart=always
User=myuser

[Install]
WantedBy=multi-user.target
```

Управление через systemctl:

- `systemctl start/stop/restart nginx` — запуск/остановка/перезапуск
- `systemctl enable/disable nginx` — добавить/убрать из автозагрузки
- `systemctl status nginx` — статус службы с последними строками лога
- `systemctl list-units --type=service` — все службы
- `systemctl daemon-reload` — перечитать unit-файлы после изменений

Journald и journalctl — системный журнал:

- `journalctl -u nginx` — логи конкретной службы
- `journalctl -f` — следить за логами в реальном времени (как tail -f)
- `journalctl --since "1 hour ago"` — логи за последний час
- `journalctl -p err` — только ошибки

Цели (targets) — аналоги runlevel: `multi-user.target` (консоль, без GUI), `graphical.target` (с GUI), `rescue.target` (однопользовательский режим).

Объяснение команд:

Лаба 1:

Лаба 2:

Лаба 3:

Лаба 4:

Лаба 5: